Talk transcript: Building quantum codes with transversal CCZ

Virginia Tech Applied Algebra Seminar, October 22, 2026. Joint work with Ken Brown. 40 minutes, then discussion.

How to use this transcript

Each section is one frame of the deck, in order. The heading gives the displayed slide number. The five unnumbered outline frames between parts have their own sections.

- **Clock** is the planned start and end time. **Budget** is the time for the frame. **Clicks** is the number of overlay steps after the first view.
- **[click]** marks where to advance. The text after it describes what has just appeared.
- *(pause)* means stop for about two seconds and let the slide be read.
- *(ask, then wait 7 seconds)* marks a question to the audience. A fallback answer follows in case nobody answers.
- Spoken text is written at about 135 words per minute. Read it aloud at a steady pace with the clock running, and mark where you run over.
- **Background (not spoken)** boxes on slides 2 to 5 collect the facts behind each sentence, with short answers to likely questions. Read them before rehearsing, not during.

Terms and notation follow the standard literature and our paper: Pauli strings X(a) and Z(b), stabilizers and stabilizer generators, X-type and Z-type stabilizers, logical operators, CSS(C_1, C_2) with $C_2 \subset C_1$, quasi-transversal, triorthogonal, column indicator. Two terms come from our paper and are defined on the slides: the signed weight and the logical label.

Winston's "How to Speak" applied to this talk

These heuristics are from Patrick Winston's MIT lecture "How to Speak" (MIT OpenCourseWare RES.TLL-005, IAP 2018). Each one is built into the script.

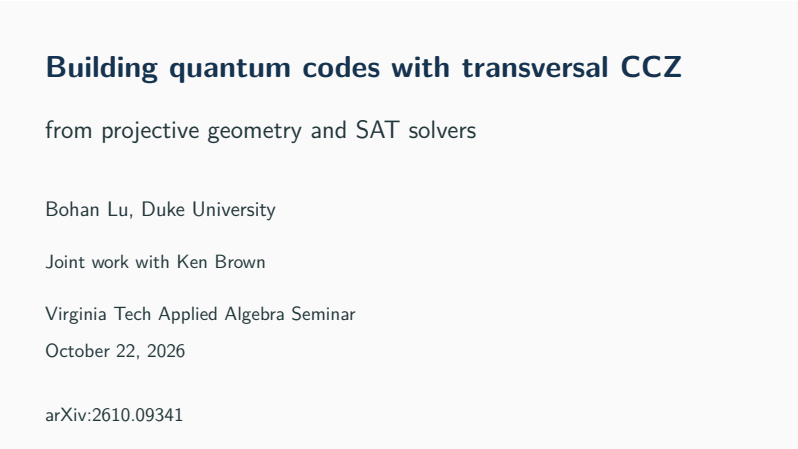
- **Empowerment promise, not a joke.** Slide 1 tells the audience what they will be able to do at the end, before 1:00. The result itself waits for slide 16, after the background that lets the audience follow it. Winston says a speaker should show both a vision and that something has been done within five minutes. The promise is the vision, and the eight-qubit cube on slides 6 to 11 shows that something has been done.
- **Cycling.** The three steps of the construction are said at slides 16, 17 and 30: "Geometry fixes the stabilizers S, a search finds the logical rows K, and linear algebra over Z_4 finds the signs."
- **Fences.** Each of these is said once, at the slide where it applies:
  - quasi-transversal CCZ versus a physical layer with no correction (slides 15 and 29)
  - the Nezami–Haah classification range $n + k \leq 38$ versus our bound $n \leq 38$ (slide 13)
  - 8-divisibility is a design choice, not part of the conditions (slide 17)
  - the 47-qubit code of Jacinto et al. is not Q48 with one point removed (slide 29).
- **Verbal punctuation.** Each outline frame says which part of six begins and what it does.
- **Ask a question.** Three planned questions, at slides 8, 10 and 20, each with a 7-second wait.
- **Collaborators first.** Ken Brown and Robert Calderbank are credited on the title slide.
- **Ending.** Slide 30 is read as contributions, and the talk ends with a salute to the audience instead of "thank you".

Pacing checkpoints

Reach this point	Clock	If about one minute behind
Outline before slide 6	6:00	On slide 6, show one coordinate face and name the other two.
Outline before slide 13	14:00	Give the history slide in two sentences. Keep the matrix/space distinction and the parity/sign distinction.
Outline before slide 22	25:00	On slide 21, give the formula, the Q48 entry and the census qualification only.
Outline before slide 27	33:00	Give the Reed–Muller argument in two sentences. Name the four stages of the bound without detail.
Slide 30	38:30	Read the contributions and two open questions. Stop at 40:00.

Do not save time by dropping the sign-feasibility distinction or the hypotheses of the lower bound. If a question interrupts a derivation, give a short answer and offer to return to it in the discussion.

Title slide

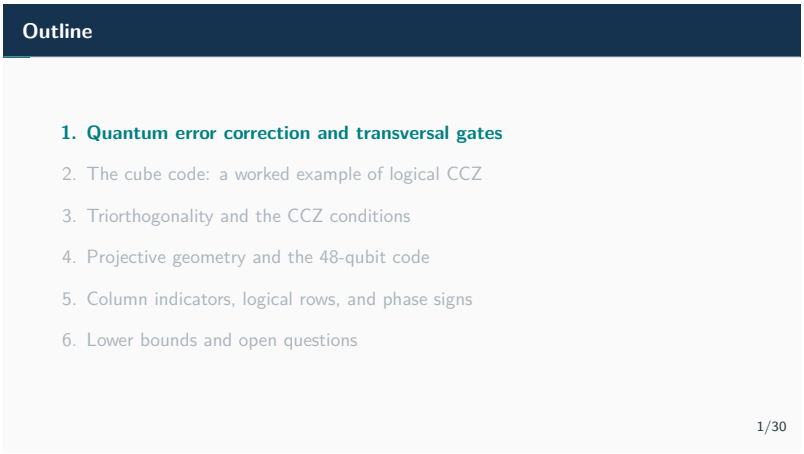


Clock 0:00–0:15 · **Budget** 15 s · **Clicks** 0

On screen: title, Bohan Lu and Ken Brown, arXiv number.

Thank you for the invitation. This is joint work with Ken Brown at Duke, and I also thank Robert Calderbank for pointers on the classical coding theory.

Slide 1. Outline

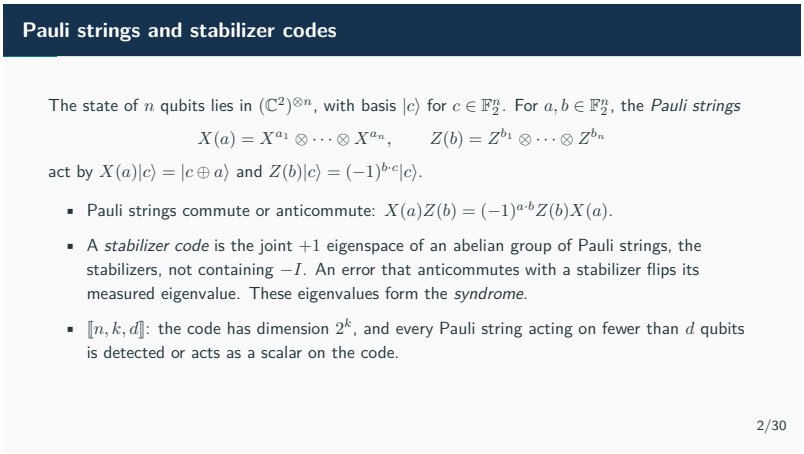


Clock 0:15–1:00 · **Budget** 45 s · **Clicks** 0

On screen: the six parts of the talk, with part 1 highlighted.

By the end of this talk you will be able to take a binary generator matrix and a choice of T or T-dagger on each qubit, and read off the logical gate from overlaps of the rows. *(pause)* We start with the quantum background we need, then work through an eight-qubit example in full. After that comes the construction of a larger code, and finally the lower bounds.

Slide 2. Pauli strings and stabilizer codes



Clock 1:00–2:00 · **Budget** 60 s · **Clicks** 3

On screen: the basis states |c⟩, and the Pauli strings X(a) and Z(b) with their action on |c⟩.

First, the quantum notation we need. The basis states of n qubits are labelled by binary vectors c of length n. For binary vectors a and b, the Pauli strings X(a) and Z(b) put an X or a Z on each qubit where a or b is one. X(a) adds a to the label, and Z(b) multiplies by minus one to the b dot c.

[click] Pauli strings commute or anticommute. X(a) and Z(b) commute when a dot b is even, and anticommute when it is odd.

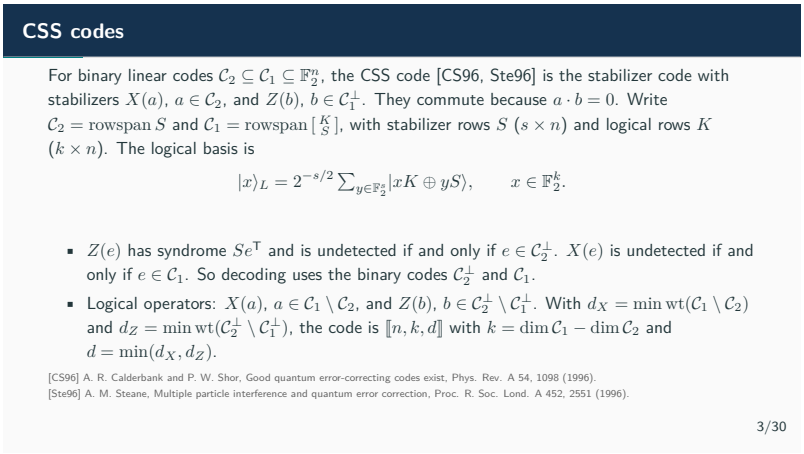
[click] A stabilizer code is the joint plus-one eigenspace of an abelian group of Pauli strings, the stabilizers, not containing minus the identity. An error that anticommutes with a stabilizer flips its measured eigenvalue, and these eigenvalues form the syndrome.

[click] An [[n,k,d]] code has dimension 2 to the k, and every Pauli string on fewer than d qubits is detected or acts as a scalar on the code.

Background (not spoken)

- **Notation.** X(a) and Z(b) with a, b ∈ F_2^n is the notation of Camps-Moreno et al. and of Rengaswamy, Calderbank and coauthors. The Pauli group is generated by ±X(a)Z(b), with Y = iXZ on a single qubit. For general strings, X(a)Z(b) and X(a′)Z(b′) commute up to the sign $(-1)^{a \cdot b' + b \cdot a'}$, the symplectic inner product. The talk only needs the case X(a) against Z(b).
- **Why Pauli strings suffice.** I, X, Z and XZ span the 2×2 complex matrices. The Knill–Laflamme conditions are linear in the error, so a code that corrects a set of Pauli errors also corrects every linear combination of them.
- **Why an anticommuting error flips the eigenvalue.** If $P|\psi\rangle = |\psi\rangle$ and $PE = -EP$, then $P(E|\psi\rangle) = -E|\psi\rangle$. So the corrupted state is in the -1 eigenspace of P.
- **Why eigenvalues are ±1.** Each stabilizer is a Hermitian Pauli string with $P^2 = I$.
- **Code dimension.** With r independent stabilizer generators the code space has dimension 2^{n-r}, so $k = n - r$.
- **Distance.** d is the smallest weight of a Pauli string that commutes with every stabilizer but is not a stabilizer up to phase. These are the nontrivial logical operators. The weight of X(a)Z(b) is the number of qubits where a or b is 1.

Slide 3. CSS codes



Clock 2:00–3:30 · **Budget** 90 s · **Clicks** 2

On screen: $C_2 \subseteq C_1$, the stabilizers X(a) and Z(b), the generator matrices S and K, and the logical basis $|x\rangle_L$.

Our codes are CSS codes, built from two nested binary codes, C2 inside C1. The stabilizers are X(a) for a in C2, and Z(b) for b in the dual of C1. They commute because a dot b is zero. We write C2 as the row space of the stabilizer rows S, and C1 as the row space of the logical rows K together with S. The logical basis state with label x is the uniform superposition of xK plus yS over all y. *(pause)* This is the formula we use for gates. A diagonal gate acts on each term of the sum separately, so every term must receive the same phase.

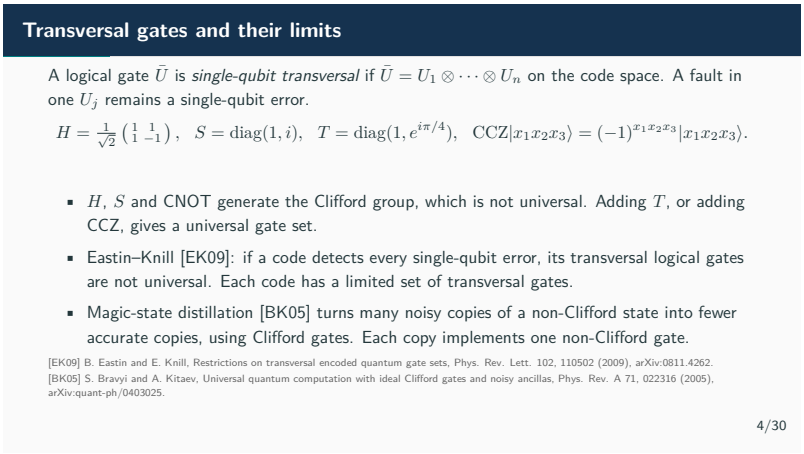
[click] A Z error Z(e) has syndrome S e-transpose, and it is undetected exactly when e is in the dual of C2. An X error X(e) is undetected exactly when e is in C1. So decoding splits into two classical problems, one for C1 and one for the dual of C2.

[click] The logical operators are X(a) with a in C1 but not in C2, and Z(b) with b in the dual of C2 but not in the dual of C1. Their minimum weights are d_X and d_Z, and d is the smaller one.

Background (not spoken)

- **Why X(a) with $a \in C_2$ fixes $|x\rangle_L$.** X(a) sends $|xK \oplus yS\rangle$ to another term of the same sum, so it only permutes the terms.
- **Why Z(b) with $b \in C_1^\perp$ fixes $|x\rangle_L$.** $Z(b)|c\rangle = (-1)^{b \cdot c}|c\rangle$, and $b \cdot c = 0$ for every c in C_1.
- **Why they commute.** $X(a)Z(b) = (-1)^{a \cdot b}\, Z(b)X(a)$, and $a \cdot b = 0$ because $a \in C_2 \subseteq C_1$ and $b \in C_1^\perp$.
- **Logical Z.** For $b \in C_2^\perp \setminus C_1^\perp$, Z(b) acts on $|x\rangle_L$ as the sign $(-1)^{b \cdot (xK)}$, which depends on x. That is why it is a nontrivial logical operator.
- **Counting.** $k = \dim C_1 - \dim C_2$. There are s X-type generators and $n - k - s$ Z-type generators, $n - k$ in all.
- **Notation in the literature.** Nielsen and Chuang, Camps-Moreno et al. and our paper write the CSS code of $C_2 \subseteq C_1$ as here. Our paper writes the logical state with column vectors, K^Tx + S^Ty. The slide uses row vectors, xK ⊕ yS, which is the same vector.
- **If asked "is every stabilizer code CSS?"** No. In a CSS code every stabilizer generator is all X or all Z. All codes in this talk are CSS.
- **If asked about the Steane code.** It has $C_2 = C_1^\perp$ (C_1 is the Hamming code), so the X-type and Z-type generators have the same supports.

Slide 4. Transversal gates and their limits



Clock 3:30–4:45 · **Budget** 75 s · **Clicks** 3

On screen: the definition of a single-qubit transversal gate, and the matrices of H, S, T and CCZ.

Now gates. A logical gate is single-qubit transversal if it is implemented by a tensor product of single-qubit gates, one on each physical qubit. If one of these gates fails, the fault stays on one qubit, and the code can handle that. Here are the gates we need: H, S, T, which is diag of 1 and e to the i pi over 4, and CCZ, which puts a minus sign on the basis state 111 and nothing else.

[click] H, S and CNOT generate the Clifford group, which is not universal. Adding T, or adding CCZ, gives a universal gate set.

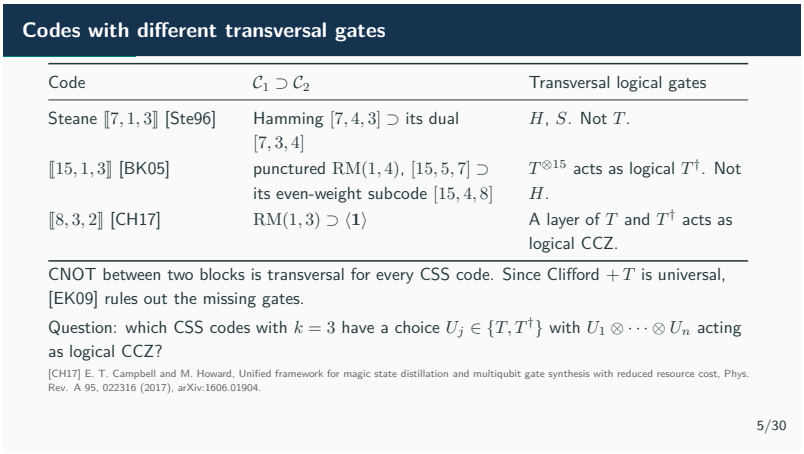
[click] Eastin and Knill proved that if a code detects every single-qubit error, its transversal logical gates are not universal. So each code has some gates transversally and lacks others.

[click] The standard way to supply a missing non-Clifford gate is magic-state distillation, introduced by Bravyi and Kitaev. Using Clifford gates, it turns many noisy copies of a non-Clifford state into fewer, more accurate copies. Each copy implements one non-Clifford gate. Distillation protocols are built from codes with transversal T or CCZ, and those are the codes we construct.

Background (not spoken)

- **Clifford group.** The unitaries that map Pauli products to Pauli products under conjugation. By the Gottesman–Knill theorem, Clifford circuits on stabilizer states can be simulated efficiently on a classical computer, so they are not universal.
- **Why T is not Clifford.** T X T† = (X + Y)/$\sqrt{2}$, which is not a Pauli product.
- **Universality.** Clifford + T generates a dense subgroup of the unitary group up to phase. CCZ with H gives Toffoli, and Clifford + CCZ is also universal.
- **The idea behind Eastin–Knill.** The transversal logical gates form a group. If the code detects every single-qubit error, a single-qubit unitary close to the identity acts trivially on the code space. So the group of transversal logical gates is finite, and a finite group cannot be dense in the unitary group.
- **Magic states.** $|\mathrm{T}\rangle = \mathrm{T}|+\rangle$ and $|\mathrm{CCZ}\rangle = \mathrm{CCZ}|{+}{+}{+}\rangle$. Gate teleportation consumes one such state, together with Clifford gates and measurement, to apply T or CCZ.
- **If asked "why CCZ rather than T?"** The standard Clifford + T circuit for CCZ uses seven T gates. Campbell and Howard distill CCZ states directly, which can lower the cost.
- **If asked about "single-qubit".** The gates $\mathrm{U_j}$ may differ from qubit to qubit. The cube code uses T on some qubits and T† on others.

Slide 5. Codes with different transversal gates



Clock 4:45–6:00 · **Budget** 75 s · **Clicks** 2

On screen: a table of three codes with their $C_1 \supset C_2$ and transversal logical gates.

Three codes show the tradeoff. The Steane code takes C1 to be the Hamming code and C2 its dual. It has transversal H and S, but not T. The fifteen-qubit code takes C1 to be punctured RM(1,4) and C2 its even-weight subcode. T on every qubit acts as logical T-dagger, but H is not transversal. The cube code, [[8,3,2]], uses RM(1,3) over the all-ones vector. There a layer of T and T-dagger gates acts as logical CCZ on the three encoded qubits.

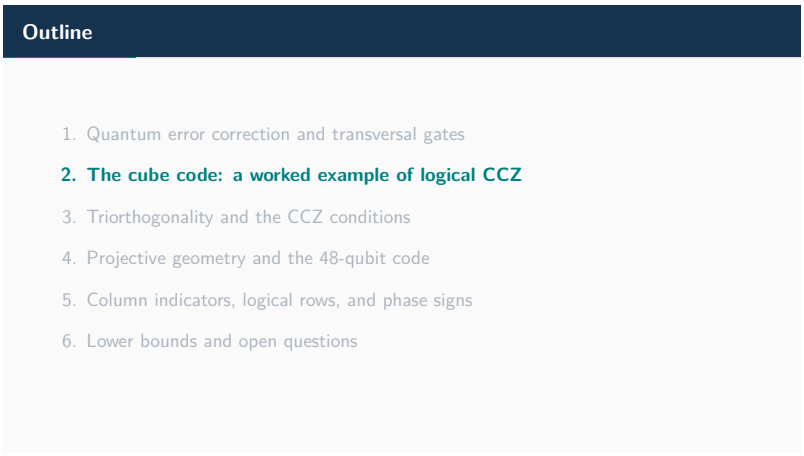
[click] CNOT between two code blocks is transversal for every CSS code. So the Steane code has the whole Clifford group transversally. Since Clifford plus T is universal, Eastin–Knill rules out T for the Steane code and H for the fifteen-qubit code.

[click] Here is the question for this talk. *(pause)* For which CSS codes with three logical qubits can we choose T or T-dagger on each qubit, so that the layer acts exactly as logical CCZ? The cube code does it with distance two. We want distance three.

Background (not spoken)

- **Steane, S gate.** The even coset C_2 has weights 0 and 4, and the odd coset has weights 3 and 7. S on every qubit multiplies $|c\rangle$ by $i^{wt(c)}$, giving 1 on the even coset and $i^3 = -i$ on the odd coset. So $S^{\otimes 7}$ is logical S†, and $(S^\dagger)^{\otimes 7}$ is logical S.
- **Steane, H gate.** H on every qubit exchanges X-type and Z-type generators. This preserves the code because $C_2 = C_1^\perp$, so both types have the same supports.
- **Fifteen-qubit code, T gate.** C_2 has weights 0 and 8, giving phase 1. The coset $\mathbf{1} \oplus C_2$ has weights 7 and 15, both $\equiv 7 \pmod 8$, giving $e^{7i\pi/4} = e^{-i\pi/4}$. So $T^{\otimes 15}$ is logical T†. Its distances are $d_X = 7$ and $d_Z = 3$.
- **Cube code.** $d_X = 4$ and $d_Z = 2$, as the next slides show.
- **If asked "why not use the fifteen-qubit code?"** It gives T on one logical qubit. We want CCZ on three logical qubits at distance 3.
- **If asked who introduced the fifteen-qubit code for distillation.** Bravyi and Kitaev used it for T distillation [BK05]. It is the quantum Reed–Muller code.

Outline before slide 6 (part 2 of 6)

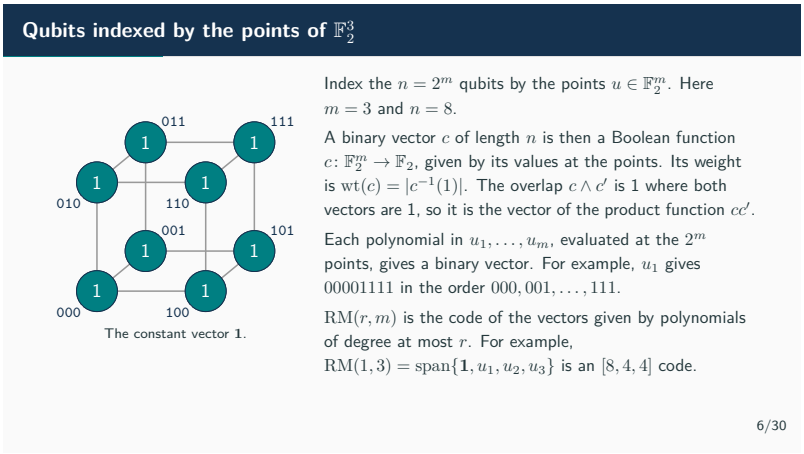


Clock 6:00–6:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 2 highlighted.

That is the background. Part two is the eight-qubit cube, worked out in full: its encoding, its logical phase, and the overlap conditions that give CCZ.

Slide 6. Qubits indexed by the points of F_2^3



Clock 6:10–7:15 · **Budget** 65 s · **Clicks** 4

On screen: the cube F_2^3. Binary vectors as Boolean functions, weight, and overlap.

Index the eight qubits by the points of F2 cubed, the vertices of this cube. A binary vector of length eight is then a Boolean function on the cube, and its weight is the number of points where it is one. The overlap of two vectors, one where both are one, is the vector of the product function.

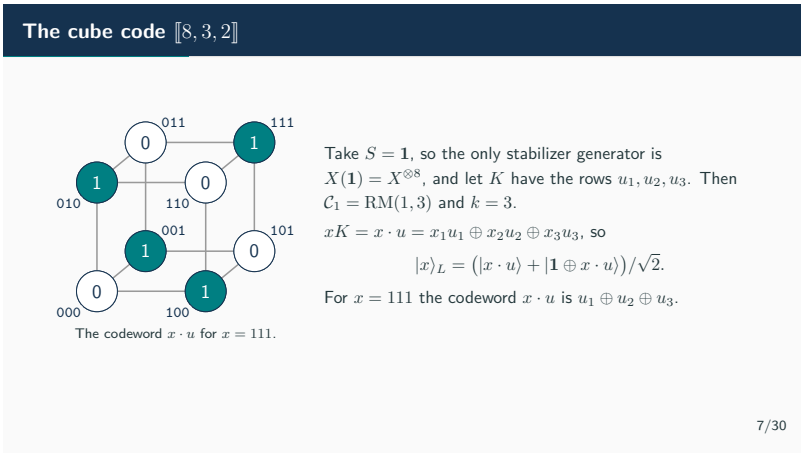
[click] Each polynomial in u1, u2, u3 gives a binary vector by evaluation at the points. Here is u1. It is one on the face where the first coordinate is one.

[click] u2 is the face in the second direction,

[click] u3 the face in the third direction,

[click] and the constant function 1 is one at all eight vertices. RM(r,m) is the code of polynomials of degree at most r. So RM(1,3), spanned by these four functions, is the [8,4,4] code of affine functions. Overlaps are now intersections. Two faces meet in an edge, and three faces meet in a vertex. We will compute the gate with exactly these pictures.

Slide 7. The cube code [[8,3,2]]



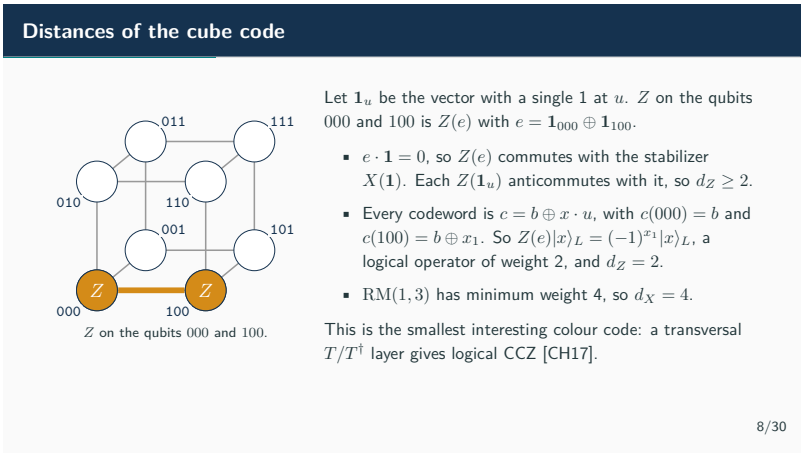
Clock 7:15–8:00 · **Budget** 45 s · **Clicks** 1

On screen: S = **1**, K = (u_1, u_2, u_3), and the logical basis state of the cube code.

Now the code. Take S to be the all-ones vector, so the only stabilizer generator is X of the all-ones vector, X on all eight qubits. Take K to be the three coordinate functions. Then C1 is RM(1,3) and k is three. The label x gives xK equal to x dot u. So each logical basis state is the truth table of x dot u plus its complement, over root two.

[click] For x equal to 111, the codeword x dot u is u1 plus u2 plus u3, shown on the cube. This is the label that CCZ will multiply by minus one.

Slide 8. Distances of the cube code



Clock 8:00–9:00 · **Budget** 60 s · **Clicks** 4

On screen: the edge from 000 to 100 with Z on both ends.

Before adding gates, let us see what this code protects. *(ask, then wait 7 seconds)* How many Z errors does it take to change a logical state of this code without being detected? *(Fallback: two, and here is an example.)* Put Z on the qubits 000 and 100. That is Z(e), where e has ones at those two points.

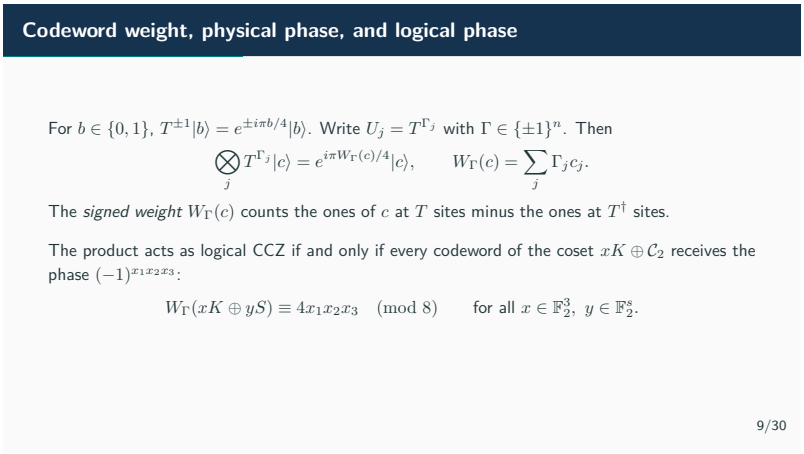
[click] The vector e has even overlap with the all-ones vector, so Z(e) commutes with the stabilizer. A single Z anticommutes with it, so d_Z is at least two.

[click] Every codeword is b plus x dot u. Its values at 000 and 100 are b and b plus x1. So Z(e) multiplies the logical state by minus one to the x1. That is a logical operator of weight two, so d_Z equals two.

[click] On the X side, RM(1,3) has minimum weight four, so d_X is four.

[click] This is the smallest interesting colour code. Its distance is only two, but a layer of T and T-dagger gives logical CCZ, and we build on that.

Slide 9. Codeword weight, physical phase, and logical phase



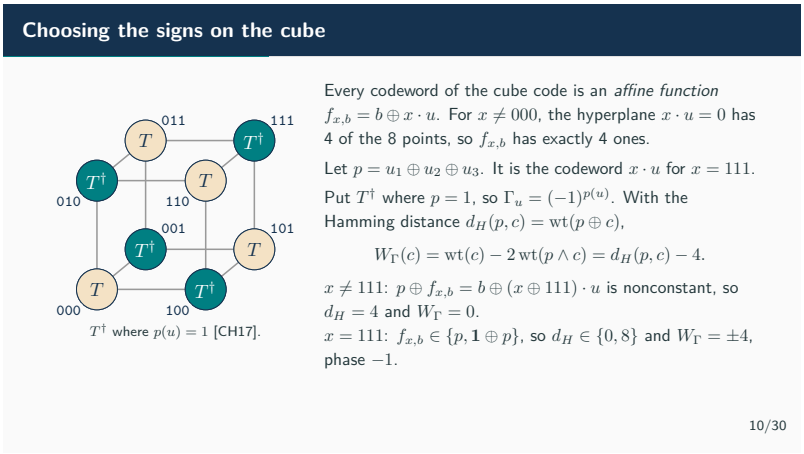
Clock 9:00–10:00 · **Budget** 60 s · **Clicks** 1

On screen: the phase of a layer of T and T† on a basis state, and the signed weight W_Γ.

Why does a layer of T gates act by a phase? T to the plus or minus one multiplies the basis state of a bit b by e to the plus or minus i pi b over four. Write the gate on qubit j as T to the Gamma$_j$, with Gamma$_j$ a sign. Then the layer multiplies the basis state c by e to the i pi W over four. Here W$_G$amma is the signed weight: the number of ones of c on T qubits minus the number on T-dagger qubits. It is an integer, not a parity.

[click] The layer acts as logical CCZ exactly when every codeword in the coset of x receives the phase minus one to the x1 x2 x3. In terms of signed weights, W$_G$amma of xK plus yS is congruent to 4 x1 x2 x3 mod 8, for all x and y. This congruence is the gate condition, and the rest of the talk rewrites it.

Slide 10. Choosing the signs on the cube



Clock 10:00–11:30 · **Budget** 90 s · **Clicks** 2

On screen: codewords as affine functions with four ones, and p = $u_1 \oplus u_2 \oplus u_3$.

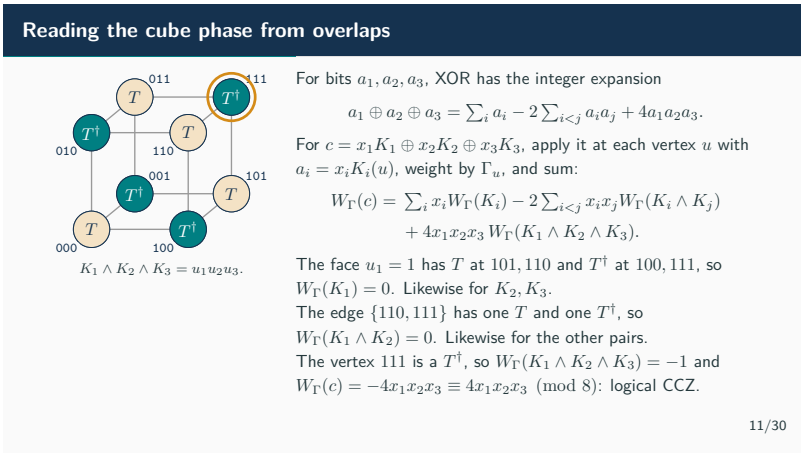
Every codeword of the cube code is an affine function, b plus x dot u. For x nonzero, the hyperplane x dot u equals zero has four of the eight points, so the codeword has exactly four ones. Let p be u1 plus u2 plus u3, the codeword of the label 111.

(ask, then wait 7 seconds) The label 111 must get a minus sign and every other label a plus sign. Where would you put the T-daggers? *(Fallback: on the four points where p is one.)*

[click] Put T-dagger where p is one and T where p is zero. Then the signed weight is the weight of c minus twice its overlap with p. Since p has weight four, this equals the Hamming distance from p to c, minus four.

[click] If x is not 111, then p plus the codeword is a nonconstant affine function. Its weight is four, so the signed weight is zero and the phase is one. If x is 111, the codeword is p or its complement, at distances zero and eight. The signed weights are minus four and plus four, and both give the phase minus one. So both terms of the logical state 111 get the same sign, and the layer is exactly CCZ.

Slide 11. Reading the cube phase from overlaps



Clock 11:30–13:15 · **Budget** 105 s · **Clicks** 3

On screen: the T/T† pattern on the cube, the integer expansion of $a_1 \oplus a_2 \oplus a_3$, and W_Γ(c) in terms of overlaps.

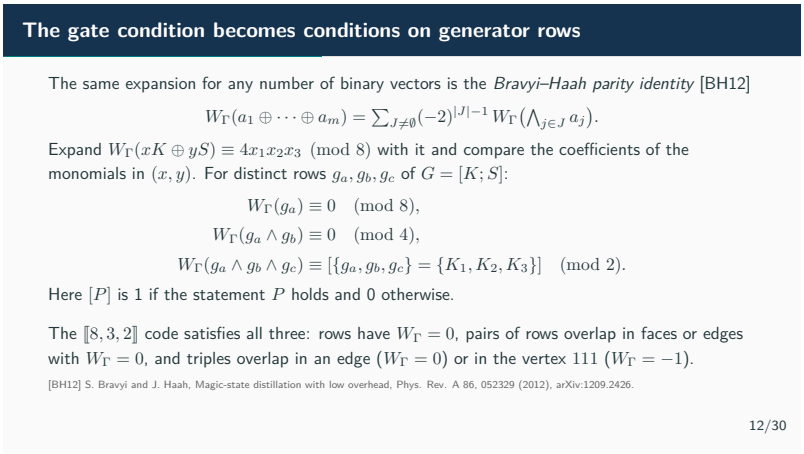
There is a second way to see this, and it is the one that generalizes. For three bits, XOR has an integer expansion: the sum, minus twice the pairwise products, plus four times the triple product. Apply it at each vertex u with a_i equal to x_i K_i of u, weight by $Gamma_u$, and sum. That gives the signed weight of c in terms of the signed weights of the rows K_i, of their pairwise overlaps, and of the triple overlap. *(pause)*

[click] K1 is u1, the face u1 equals 1. It has T at 101 and 110, and T-dagger at 100 and 111. So its signed weight is zero, and the same holds for K2 and K3.

[click] K1 and K2 overlap in the edge 110, 111. That is one T and one T-dagger, so again zero, and likewise for the other pairs.

[click] All three overlap in the vertex 111, which carries T-dagger. Its signed weight is minus one. So $W_G$amma of c is minus 4 x1 x2 x3, which is 4 x1 x2 x3 mod 8. That is CCZ, read off a face, an edge and a vertex. The stabilizer row adds terms of the same kind, a face and an edge, and they vanish too.

Slide 12. The gate condition becomes conditions on generator rows



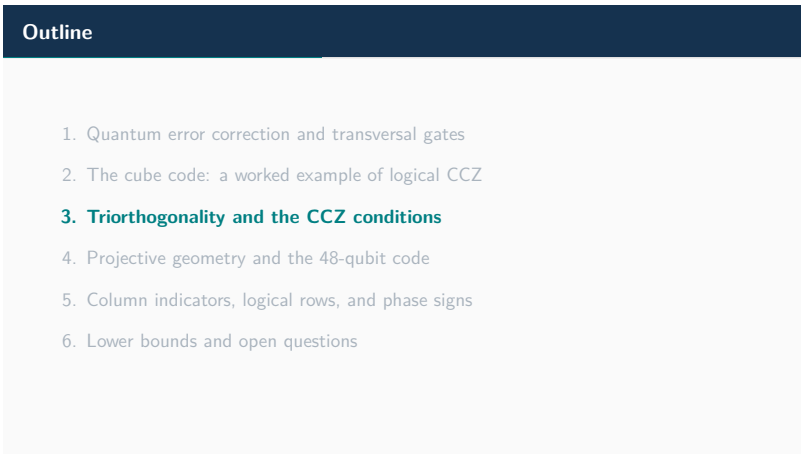
Clock 13:15–14:00 · **Budget** 45 s · **Clicks** 1

On screen: the Bravyi–Haah parity identity, and the three congruences for single rows, pairs and triples.

The same expansion holds for any number of vectors. This is the Bravyi–Haah parity identity: an overlap of t vectors has coefficient minus two to the power t minus one. Expand the gate condition this way and compare coefficients. Single rows need signed weight zero mod 8. Pair overlaps carry minus two, so they need zero mod 4. Triple overlaps carry four, so only their parity matters, and it must be odd exactly for the triple K1, K2, K3. Overlaps of four or more rows carry a multiple of eight and drop out.

[click] The cube code satisfies all three. These are the conditions we impose on larger codes.

Outline before slide 13 (part 3 of 6)

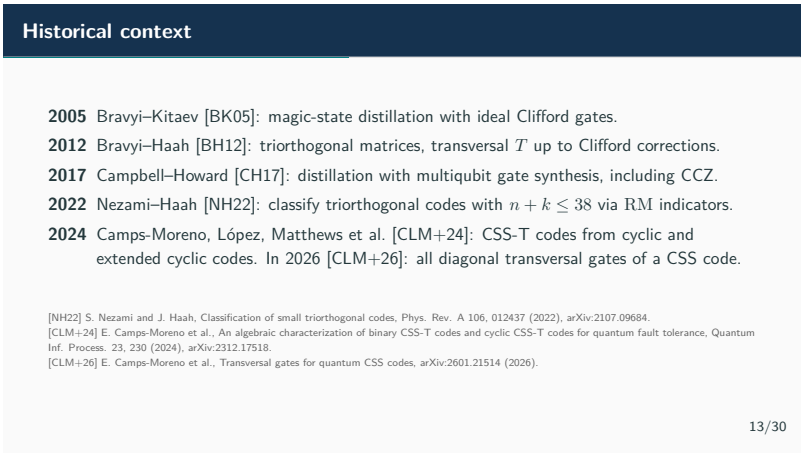


Clock 14:00–14:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 3 highlighted.

Part three. The cube gave exact signed conditions. Now I relate them to triorthogonality, and separate the parity conditions from the choice of signs.

Slide 13. Historical context

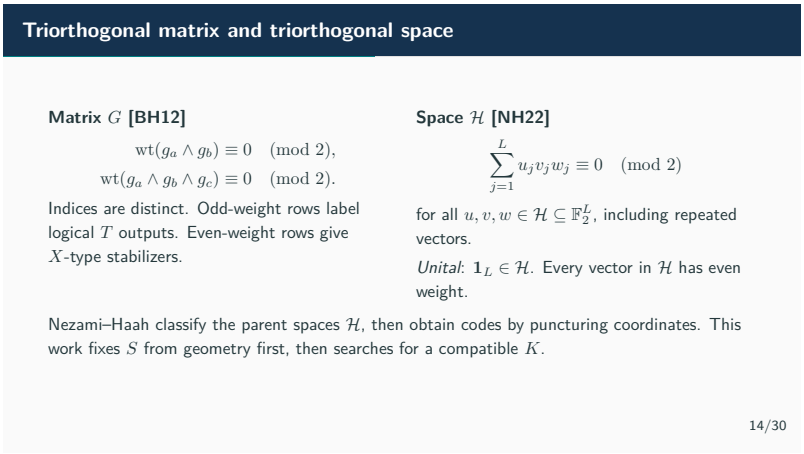


Clock 14:10–14:45 · **Budget** 35 s · **Clicks** 0

On screen: a timeline from 2005 to 2026.

Briefly, the history. Bravyi and Kitaev introduced magic-state distillation. Bravyi and Haah wrote transversal T as parity conditions on binary rows, triorthogonal matrices. Campbell and Howard gave the CCZ version, which we use. Nezami and Haah classified triorthogonal codes with n plus k at most 38. That is a range for T codes, and it is a different statement from our bound n at most 38. And Camps-Moreno, López, Matthews and coauthors study CSS-T codes, and in 2026 all diagonal transversal gates of a CSS code.

Slide 14. Triorthogonal matrix and triorthogonal space

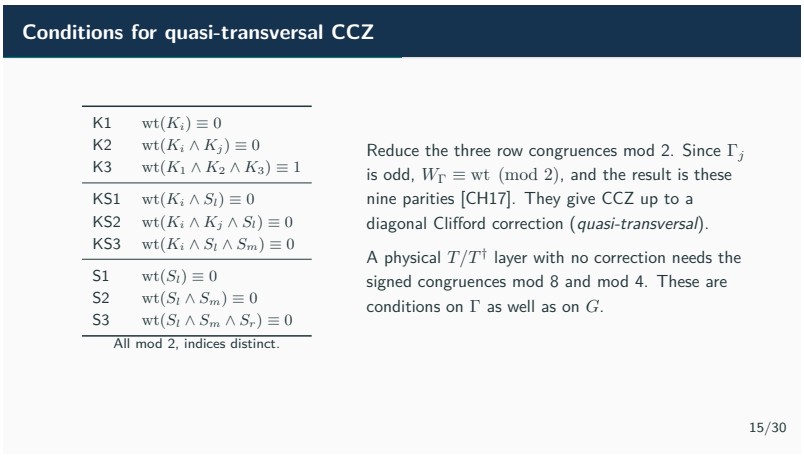


Clock 14:45–16:00 · **Budget** 75 s · **Clicks** 0

On screen: the Bravyi–Haah matrix conditions beside the Nezami–Haah space condition.

There are two related definitions, and they are easy to confuse. On the left is the triorthogonal matrix of Bravyi and Haah. Pair overlaps and triple overlaps of distinct rows are even. Odd-weight rows give the logical T outputs, and even-weight rows give the X-type stabilizers. On the right is the triorthogonal space of Nezami and Haah. The triple sum vanishes for all u, v, w in the space, including repeated vectors. Taking u, v and w equal shows every vector has even weight. Unital means the all-ones vector is in the space. So the row span of a triorthogonal matrix with odd rows is not a triorthogonal space. *(pause)* Nezami and Haah classify the parent spaces and obtain codes by puncturing coordinates. We go the other way. We fix S from geometry first, then search for a compatible K. And CCZ needs one odd triple, K1, K2, K3, which neither definition allows. That is why we need the generalized conditions on the next slide.

Slide 15. Conditions for quasi-transversal CCZ



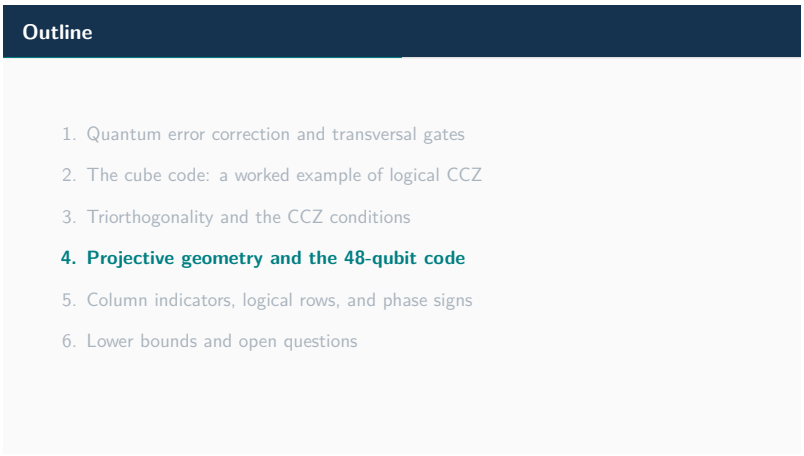
Clock 16:00–17:30 · **Budget** 90 s · **Clicks** 1

On screen: the nine parity conditions K1–K3, KS1–KS3 and S1–S3.

Reduce the three signed conditions mod 2. Each Gamma$_j$ is odd, so a signed weight has the same parity as the ordinary weight. What remains are these nine parity conditions of Campbell and Howard. K1 to K3 involve only logical rows, KS1 to KS3 mix logical and stabilizer rows, and S1 to S3 involve only stabilizer rows. K3 is the only odd condition, and it selects CCZ. These nine conditions give CCZ up to a diagonal Clifford correction. This is called quasi-transversal CCZ.

[click] Here is a distinction I want to keep clear. *(pause)* A physical layer of T and T-dagger with no correction needs more, namely the signed congruences mod 8 and mod 4. Those are conditions on the signs Gamma as well as on the matrix. A code can satisfy all nine parities and still have no valid choice of signs. We will see an example at length 47.

Outline before slide 16 (part 4 of 6)

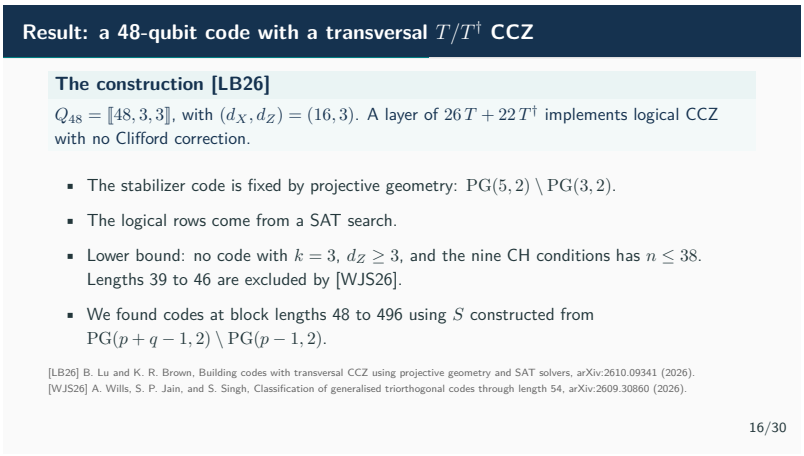


Clock 17:30–17:40 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 4 highlighted.

Part four, the construction. We now build the 48-qubit code, starting from the geometry that fixes its stabilizers.

Slide 16. Result: a 48-qubit code with a transversal T/T† CCZ

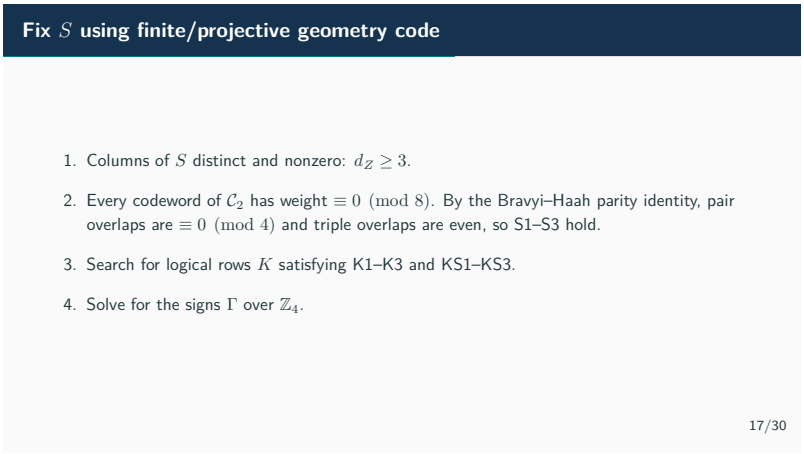


Clock 17:40–19:00 · **Budget** 80 s · **Clicks** 0

On screen: Q48 = [[48,3,3]] with (d_X, d_Z) = (16, 3), and four bullets on the construction and the bounds.

Here is the result. Q48 is a [[48,3,3]] code, with d_X equal to 16 and d_Z equal to 3. A layer of 26 T gates and 22 T-dagger gates implements logical CCZ, with no Clifford correction. It is built in three steps. Geometry fixes the stabilizers S: the stabilizer code comes from PG(5,2) minus PG(3,2). A SAT search finds the logical rows K. And linear algebra over Z4 finds the signs. *(pause)* For the lower bound, our theorem says that no code with three logical qubits, d_Z at least 3, and the nine CH conditions has n at most 38. Lengths 39 to 46 are excluded by the classification of Wills, Jain and Singh. And other subspace complements give codes at block lengths from 48 up to 496.

Slide 17. Fix S using finite/projective geometry code



Clock 19:00–20:00 · **Budget** 60 s · **Clicks** 3

On screen: step 1, distinct nonzero columns of S give $d_Z \geq 3$.

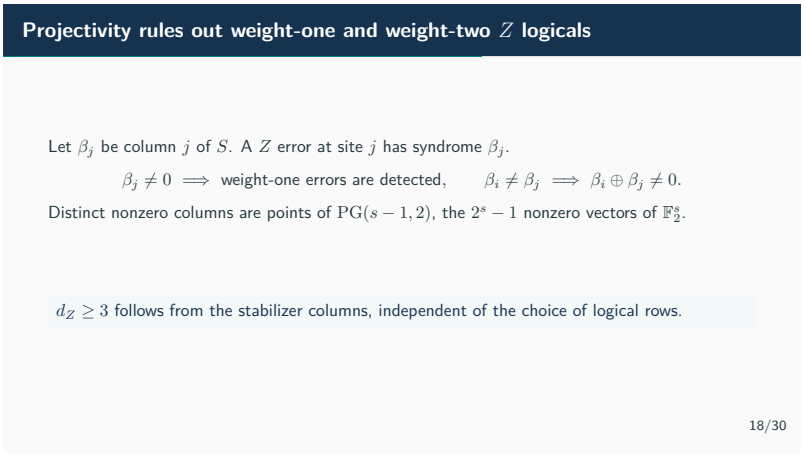
The construction has four steps. First, choose S with distinct nonzero columns. That alone gives d_Z at least three, as the next slide shows.

[click] Second, require every codeword of C2 to have weight divisible by 8. By the Bravyi–Haah parity identity, pair overlaps are then zero mod 4 and triple overlaps are even, so S1 to S3 hold. This divisibility is our design choice. The CH conditions do not require it.

[click] Third, search for logical rows K that satisfy K1 to K3 and KS1 to KS3. This is the only search.

[click] Fourth, solve for the signs Gamma over Z4. That is a linear-algebra test.

Slide 18. Projectivity rules out weight-one and weight-two Z logicals

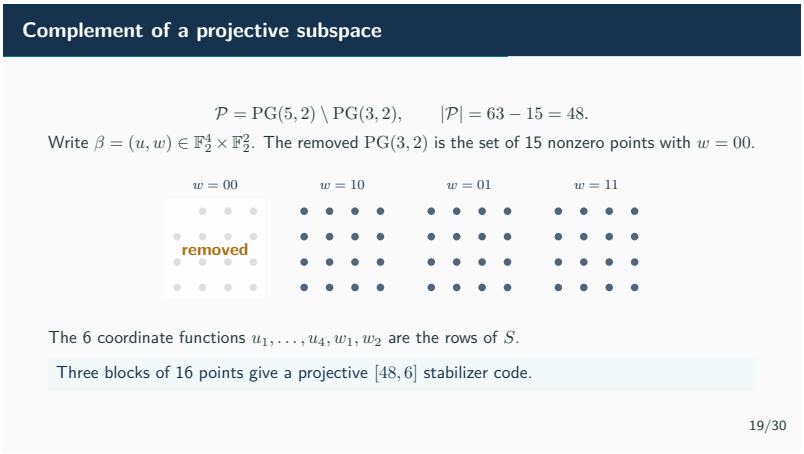


Clock 20:00–21:00 · **Budget** 60 s · **Clicks** 0

On screen: the syndrome of a Z error is a column β_j of S, and distinct nonzero columns detect one or two Z errors.

Why distinct nonzero columns? Let beta_j be column j of S. A Z error on qubit j has syndrome beta_j. If no column is zero, every single Z error is detected. If the columns are distinct, beta_i plus beta_j is nonzero, so every pair of Z errors is detected too. So d_Z is at least three. *(pause)* Distinct nonzero columns are points of the projective space PG(s minus 1, 2), the nonzero vectors of F2 to the s. This is the parity-check argument for a classical code of distance three. It does not involve the logical rows, and it covers d_Z only. We compute d_X for the final code.

Slide 19. Complement of a projective subspace



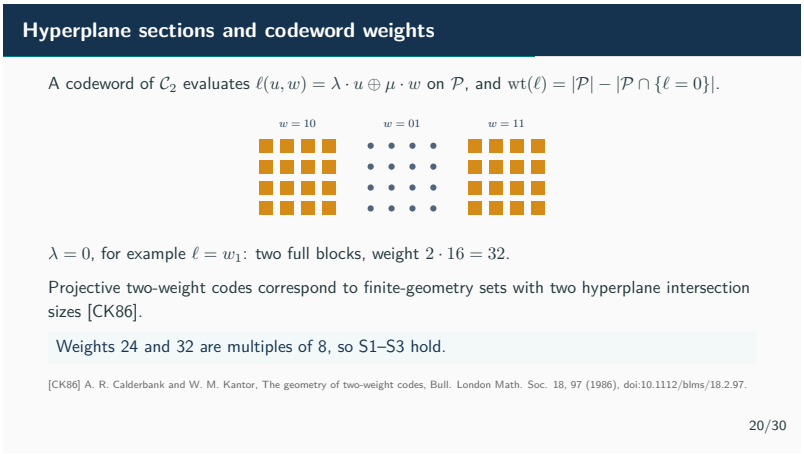
Clock 21:00–22:00 · **Budget** 60 s · **Clicks** 1

On screen: P = PG(5,2) \ PG(3,2) with 63 − 15 = 48 points, drawn as four blocks indexed by w.

Take the 63 points of PG(5,2) and remove a PG(3,2), which has 15 points. That leaves 48. Write a point as beta equals u, w, with u in F2 to the 4 and w in F2 squared. The removed subspace is the 15 nonzero points with w equal to 00. In the picture, the four blocks are the values of w. Inside each block the rows are u1 u2 and the columns are u3 u4, in Gray order.

[click] Remove the block w equals 00. The six coordinate functions u1 to u4, w1 and w2, evaluated on the 48 points, are the rows of S. Three blocks of 16 points give a projective [48,6] stabilizer code. We keep this picture for the weights, the logical rows and the signs.

Slide 20. Hyperplane sections and codeword weights



Clock 22:00–23:30 · **Budget** 90 s · **Clicks** 1

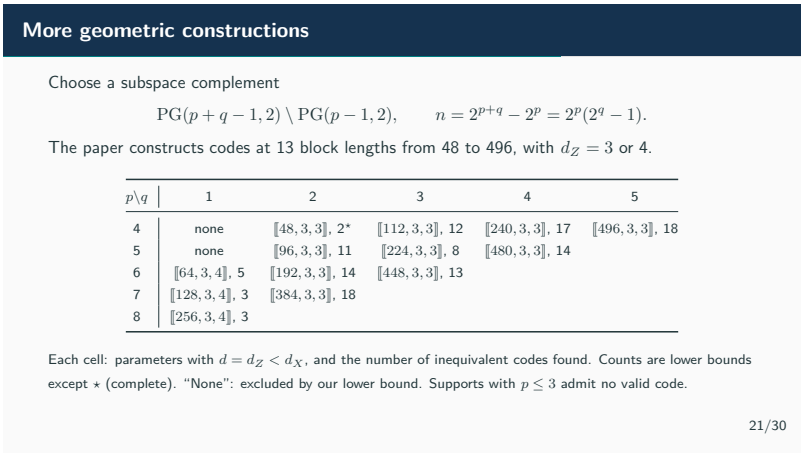
On screen: a codeword of C_2 as a linear function ℓ on the 48 points, with u_1 marked in gold.

Now the weights. A codeword of C2 is a linear function, lambda dot u plus mu dot w, evaluated on the 48 points. Its weight is 48 minus the number of points on the hyperplane where it vanishes. If lambda is nonzero, for example u1, the gold squares show that it is one on eight points of each block. So the weight is 3 times 8, which is 24.

(ask, then wait 7 seconds) What is the weight of w1? *(Fallback: click and read it off.)*

[click] w1 is one on two full blocks, so its weight is 2 times 16, which is 32. These are the only two cases. So the nonzero weights are 24 and 32, both multiples of 8, and S1 to S3 hold. *(pause)* A projective code with two nonzero weights is a two-weight code. Calderbank and Kantor relate these codes to point sets in finite geometry with two intersection sizes with hyperplanes.

Slide 21. More geometric constructions



More geometric constructions

Choose a subspace complement

$$\mathrm{PG}(p+q-1,2) \setminus \mathrm{PG}(p-1,2), \qquad n = 2^{p+q} - 2^p = 2^p(2^q-1).$$

The paper constructs codes at 13 block lengths from 48 to 496, with $d_Z = 3$ or 4.

$p\backslash q$	1	2	3	4	5
4	none	$[\![48,3,3]\!]$, 2*	$[\![112,3,3]\!]$, 12	$[\![240,3,3]\!]$, 17	$[\![496,3,3]\!]$, 18
5	none	$[\![96,3,3]\!]$, 11	$[\![224,3,3]\!]$, 8	$[\![480,3,3]\!]$, 14	
6	$[\![64,3,4]\!]$, 5	$[\![192,3,3]\!]$, 14	$[\![448,3,3]\!]$, 13		
7	$[\![128,3,4]\!]$, 3	$[\![384,3,3]\!]$, 18			
8	$[\![256,3,4]\!]$, 3				

Each cell: parameters with $d = d_Z < d_X$, and the number of inequivalent codes found. Counts are lower bounds except $\star$ (complete). "None": excluded by our lower bound. Supports with $p \leq 3$ admit no valid code.

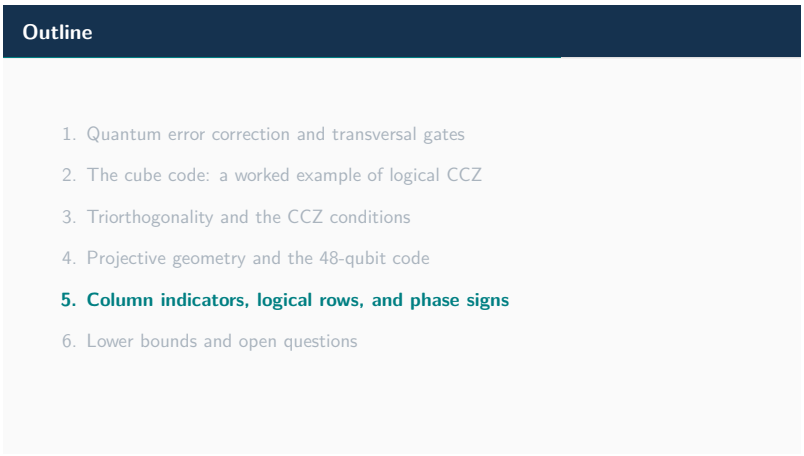
21/30

Clock 23:30–25:00 · **Budget** 90 s · **Clicks** 0

On screen: n = $2^p(2^q - 1)$ for PG(p+q−1,2) \ PG(p−1,2), and a table of codes found for p = 4 to 8 and q = 1 to 5.

The same idea works for any subspace complement, PG(p plus q minus 1, 2) minus PG(p minus 1, 2). The length is 2 to the p times 2 to the q minus 1. Q48 is the case p equals 4, q equals 2. The paper constructs codes at 13 block lengths from 48 to 496, with d_Z equal to three or four. Each cell gives the parameters and the number of inequivalent codes we found. For example, p equal to 6 and q equal to 1 gives 64 qubits with d_Z equal to four. *(pause)* Two qualifications. Only the starred entry at 48 is a complete census for that support. The other counts are lower bounds. The entries marked "none" are excluded by our lower bound, and supports with p at most 3 admit no valid code. This is a family of finite constructions, not an asymptotic statement.

Outline before slide 22 (part 5 of 6)

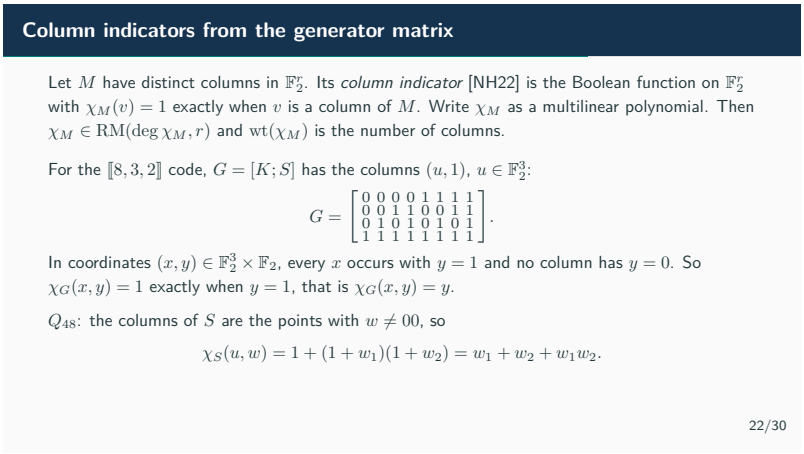


Clock 25:00–25:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 5 highlighted.

Part five. Geometry has fixed the stabilizers. Now we find the logical rows using column indicators, and then solve for the signs.

Slide 22. Column indicators from the generator matrix



Clock 25:10–26:30 · **Budget** 80 s · **Clicks** 2

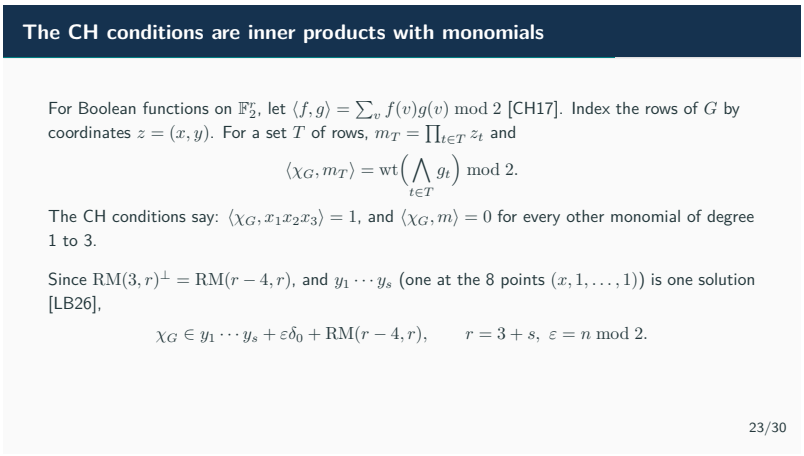
On screen: the definition of the column indicator χ_M.

To find the logical rows we switch from rows to columns. Let M have distinct columns in F2 to the r. Its column indicator, as in Nezami and Haah, is the Boolean function chi_M on F2 to the r that is one exactly at the columns of M. Written as a polynomial, it lies in a Reed–Muller code, and its weight is the number of columns.

[click] For the cube, G stacks K on S, so its columns are u followed by 1, for every u in F2 cubed. In coordinates x, y, every x occurs with y equal to 1, and no column has y equal to 0. So chi_G is just y.

[click] For Q48, the columns of S are the points with w not equal to 00. So chi_S is 1 plus, 1 plus w1, times, 1 plus w2, which is w1 plus w2 plus w1 w2. It has degree two and does not depend on u.

Slide 23. The CH conditions are inner products with monomials



Clock 26:30–28:00 · **Budget** 90 s · **Clicks** 2

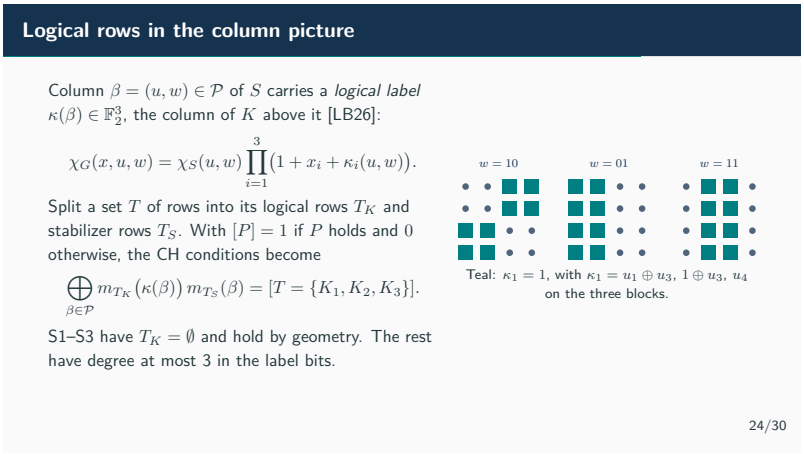
On screen: the inner product ⟨f, g⟩ and the identity $\langle \chi_G, m_T\rangle = \mathrm{wt}(\wedge_{t\in T} g_t) \bmod 2$.

Why columns? Because overlaps become inner products. For Boolean functions on F2 to the r, the inner product is the sum of f times g, mod 2. Index the rows of G by coordinates z. For a set T of rows, let m_T be the monomial, the product of those coordinates. Then the inner product of chi_G with m_T is the weight of the overlap of those rows, mod 2. Summing over the columns is summing over the points where chi_G is one.

[click] So the CH conditions say this: chi_G has inner product one with x1 x2 x3, and zero with every other monomial of degree one to three.

[click] The dual of RM(3,r) is RM(r minus 4, r). The product y1 through y_s is one solution. It is one at the eight points x, 1, ..., 1, and among these monomials only x1 x2 x3 has odd inner product with it. So every valid chi_G lies in this solution, plus epsilon delta-zero, plus RM(r minus 4, r). Here delta-zero is the indicator of the origin and epsilon is n mod 2. That coset statement is a lemma in our paper.

Slide 24. Logical rows in the column picture



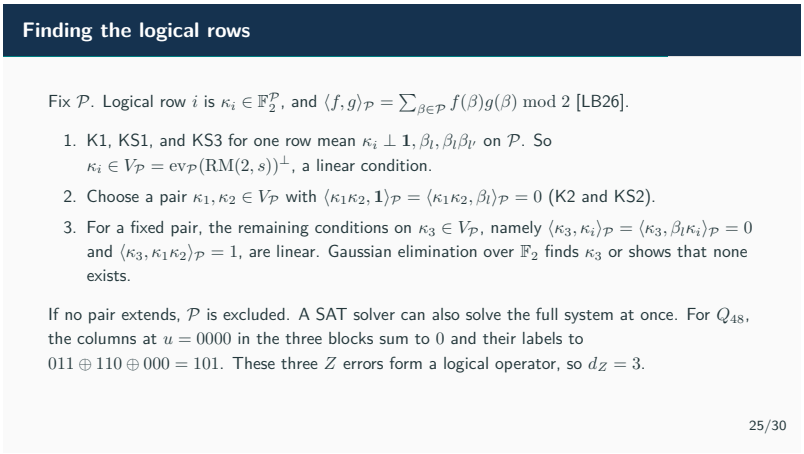
Clock 28:00–29:30 · **Budget** 90 s · **Clicks** 1

On screen: the logical label κ(β), the factorization of χ_G, and the CH conditions as sums over the 48 points.

For Q48, S is fixed, so the unknowns are the logical rows. In the column picture, each column beta of S carries a logical label kappa of beta in F2 cubed, the column of K above it. Then chi_G factors as chi_S times a product that forces x to equal kappa at each point. Split a set T of rows into its logical rows T_K and stabilizer rows T_S. The CH conditions become a sum over the 48 points, which must equal one exactly for the logical triple. S1 to S3 have no logical rows, and the geometry already gave them. The rest are equations of degree at most three in the 144 label bits.

[click] Here is the first logical row of Q48 on the support. kappa_1 is u1 plus u3 on the first block, 1 plus u3 on the second, and u4 on the third. It has eight ones in each block.

Slide 25. Finding the logical rows



Clock 29:30–31:30 · **Budget** 120 s · **Clicks** 3

On screen: step 1 of Algorithm 1, $\kappa_i \in V_P = \mathrm{ev}_P(\mathrm{RM}(2,s))^{\perp}$.

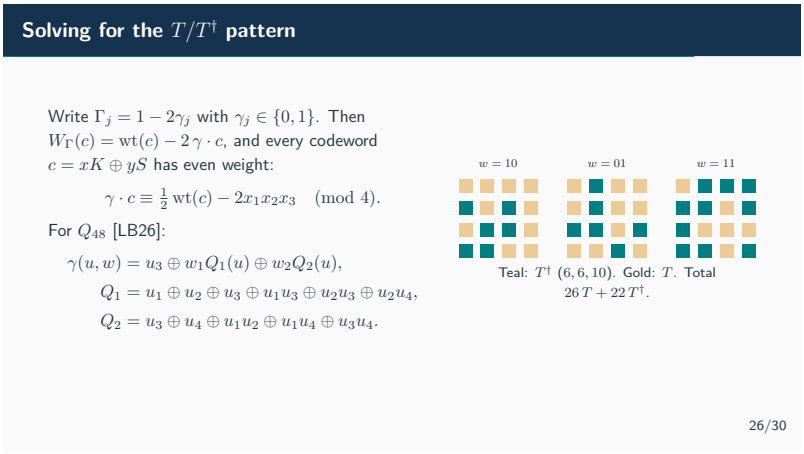
How do we solve these equations? Fix the support P. Logical row i is a function kappa_i on P, and the inner product is summed over P. This is Algorithm 1 in the paper. Step one: K1, KS1 and KS3 for a single row say that kappa_i is orthogonal to 1, to each beta_l, and to each product beta_l $\mathrm{beta}_{l'}$ prime. Those functions span RM(2,s). So kappa_i lies in V_P, the dual of RM(2,s) evaluated on P, which is a linear condition.

[click] Step two: choose a pair kappa_1, kappa_2 in V_P whose product is orthogonal to 1 and to every beta_l. Those are the conditions K2 and KS2.

[click] Step three: once the pair is fixed, every remaining condition on kappa_3 is linear. It must be orthogonal to kappa_i and to beta_l kappa_i, and have inner product one with kappa_1 kappa_2. Gaussian elimination over F2 finds kappa_3 or proves that none exists.

[click] If no pair extends, the support is excluded. A SAT solver can also solve the whole system at once. *(pause)* For Q48, take the three columns at u equal to 0000 in the three blocks. They sum to zero, so Z on these three qubits commutes with every X-type stabilizer. Their labels sum to 011 plus 110 plus 000, which is 101, not zero. So these three Z's form a logical operator, and d_Z is exactly three.

Slide 26. Solving for the T/T† pattern



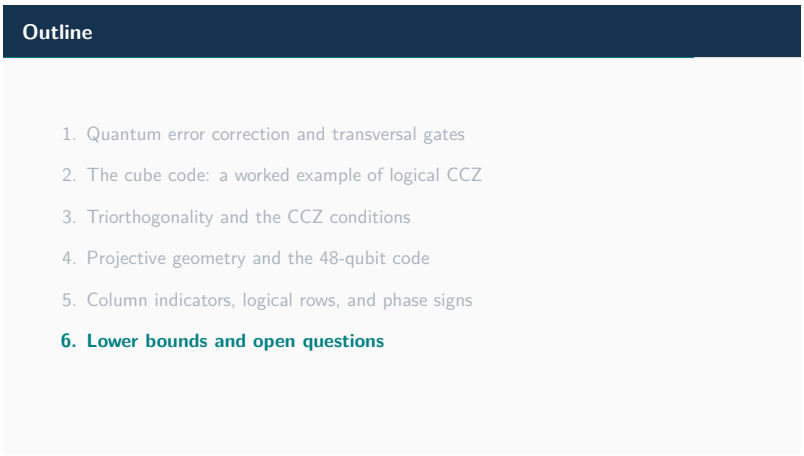
Clock 31:30–33:00 · **Budget** 90 s · **Clicks** 1

On screen: $\Gamma_j = 1 - 2\gamma_j$ and the congruence $\gamma \cdot c \equiv \frac{1}{2}\mathrm{wt}(c) - 2x_1x_2x_3 \pmod 4$.

The last step is the signs. Write Gamma$_j$ as 1 minus 2 gamma$_j$, with gamma$_j$ a bit that is one where we put T-dagger. Then the signed weight is the weight minus twice gamma dot c, computed as an integer. Every codeword has even weight, so we can divide the gate condition by two. Gamma dot c must be congruent to half the weight of c, minus 2 x1 x2 x3, mod 4. That is linear in gamma over Z4, with the extra requirement that gamma is binary. Elimination over Z4 decides whether a solution exists.

[click] For Q48, here is the solution from the paper. Gamma is u3 plus w1 Q1 of u plus w2 Q2 of u, with these two quadratic functions. In the picture, teal is T-dagger, with 6, 6 and 10 in the three blocks, 22 in all. Gold is T, 26 in all. These are coordinate positions, not a hardware layout.

Outline before slide 27 (part 6 of 6)

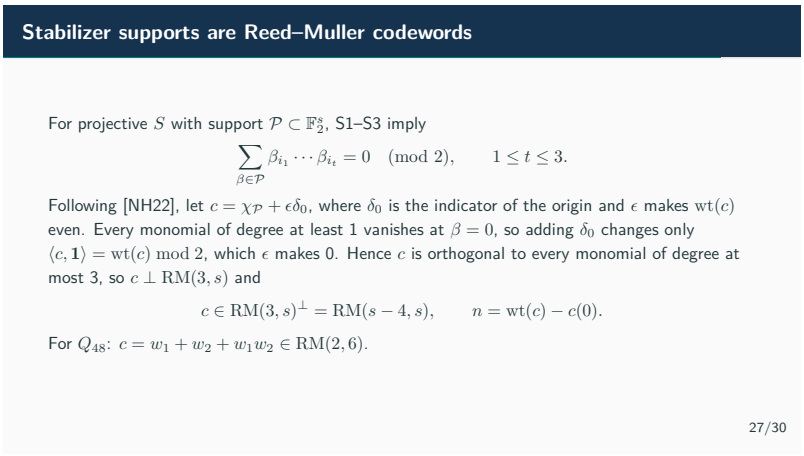


Clock 33:00–33:10 · **Budget** 10 s · **Clicks** 0

On screen: the outline, with part 6 highlighted.

Part six, the last part. The construction is complete. Now the same Boolean view explains the lower bound and what remains open.

Slide 27. Stabilizer supports are Reed–Muller codewords

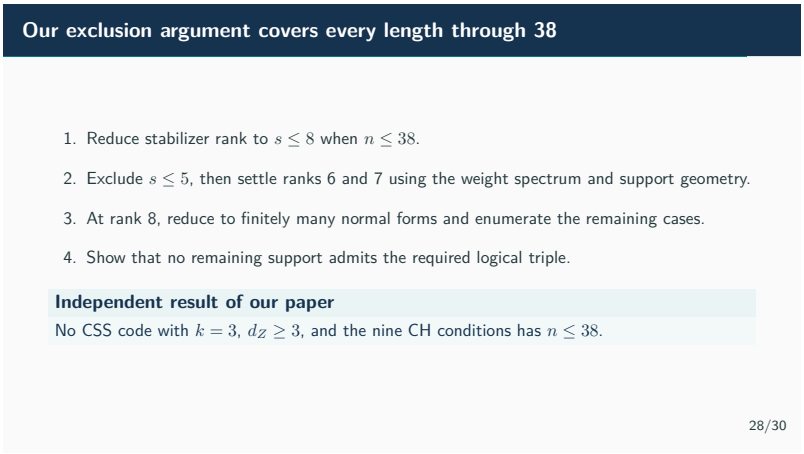


Clock 33:10–35:00 · **Budget** 110 s · **Clicks** 0

On screen: S1–S3 as vanishing sums of monomials over P, and c = χ_P + $\varepsilon\delta_0$ ∈ RM(s−4, s).

Why can no short code work? Take a projective S with support P in F2 to the s. S1 to S3 say that the sum over P of every monomial of degree one to three in the coordinates vanishes mod 2. Following Nezami and Haah, let c be the indicator of P plus epsilon times the indicator of the origin, with epsilon chosen to make the weight of c even. Every monomial of positive degree vanishes at the origin. So adding the origin changes only the inner product with 1, which is the parity of the weight, and epsilon makes it zero. *(pause)* Hence c is orthogonal to every monomial of degree at most three, so c is orthogonal to RM(3,s), and c lies in RM(s minus 4, s). The block length is the weight of c minus c at the origin. For Q48, c is w1 plus w2 plus w1 w2, a codeword of RM(2,6). So a short code needs a low-weight codeword of a Reed–Muller code. Low-weight codewords of Reed–Muller codes are classified, and that is what makes the search below length 39 finite.

Slide 28. Our exclusion argument covers every length through 38

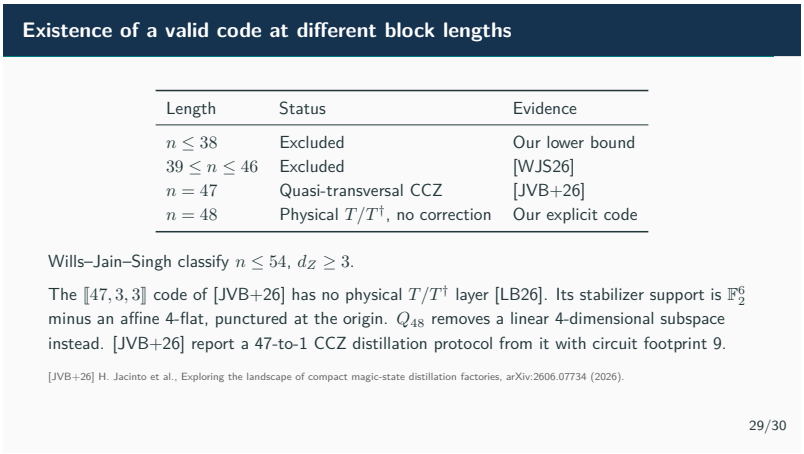


Clock 35:00–36:30 · **Budget** 90 s · **Clicks** 0

On screen: four proof stages, and the theorem: no CSS code with k = 3, $d_Z \geq 3$ and the nine CH conditions has n ≤ 38.

The proof that nothing works through length 38 has four stages. First, reduce to stabilizer rank s at most 8 when n is at most 38. Second, exclude s at most 5, then settle ranks 6 and 7 using the weight distribution and the geometry of the support. Third, at rank 8, reduce to finitely many normal forms and enumerate what remains. Fourth, show that no remaining support admits the required logical triple. *(pause)* The result is our theorem. No CSS code with k equal to 3, d_Z at least 3 and the nine CH conditions has n at most 38. Two remarks. The rank-8 coverage and the logical-row exclusions are complete, not sampled. And because the theorem assumes only the nine parity conditions, it rules out quasi-transversal codes as well as physical layers.

Slide 29. Existence of a valid code at different block lengths

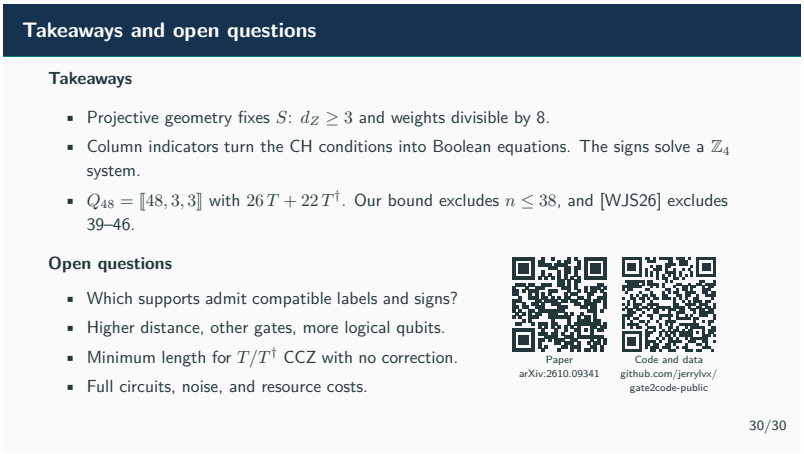


Clock 36:30–38:30 · **Budget** 120 s · **Clicks** 0

On screen: a table of lengths, status and evidence, and three sentences on the [[47,3,3]] code.

Here is the situation by length. Up to 38 is excluded by our bound. 39 to 46 is excluded by Wills, Jain and Singh, who classify generalized triorthogonal codes up to length 54 with d_Z at least 3. At 47 there is a quasi-transversal CCZ code, the [[47,3,3]] code of Jacinto and coauthors. At 48 we have Q48, with a physical layer and no correction. *(pause)* Now the distinction between 47 and 48. The 47-qubit code satisfies the nine parities, but it has no physical layer of T and T-dagger. We show this in our paper. So at 47 a Clifford correction is needed. That statement is about this specific code. It does not show that 48 is the minimum length for a physical layer, and it does not show that the 47-qubit code is unique. It is also not Q48 with one point removed. Its stabilizer support is F2 to the 6 minus an affine 4-flat, with the origin removed. Q48 removes a linear 4-dimensional subspace. The stabilizer weights differ as well: 16, 24 and 32 for the 47-qubit code, and 23, 24, 31 and 32 for any puncture of Q48. Finally, Jacinto and coauthors report a 47-to-1 CCZ distillation protocol from their code with circuit footprint 9. That number is theirs, and it is about the circuit, not the code length.

Slide 30. Takeaways and open questions



Clock 38:30–40:00 · **Budget** 90 s · **Clicks** 0

On screen: three takeaways, four open questions, and QR codes for the paper and the code.

Here is what we contributed. *(pause)* Projective geometry fixes S, giving d_Z at least three and weights divisible by eight. Column indicators turn the CH conditions into Boolean equations for the logical rows, and the signs solve a linear system over Z4. Together these give Q48, a [[48,3,3]] code in which 26 T and 22 T-dagger gates act as logical CCZ. Our bound excludes lengths up to 38, and Wills, Jain and Singh exclude 39 to 46. In one sentence: geometry fixes the stabilizers, a search finds the logical rows, and linear algebra over Z4 finds the signs. *(pause)* Some open questions. Which supports admit compatible logical rows and signs? Can we reach higher distance, other gates, or more logical qubits? What is the minimum length for a CCZ layer of T and T-dagger with no correction? And what do full circuits cost once noise is included? If you work on divisible codes, or on completing a code from a fixed support, I would like to hear your ideas. The QR codes link to the paper and the code. *(salute, then stop)* It has been a pleasure to present this work to algebraists and coding theorists here at Virginia Tech. I look forward to your questions.

After the talk

Leave slide 30 on screen during questions. Prepared answers to likely technical questions are in questions.md.